JAVA SDE-2 INTERVIEW PREPARATION SERIES

SYSTEM DESIGN AND BACKEND - BOOK 11 OF 14 - STUDY STEP 19L

Spring AI for Java Engineers

Model Clients, Prompts, Structured Output, Retrieval, Tools, Evaluation, and Operations

BY

Vinay Reddy Kalluri

A roadmap for adding generative-AI capabilities to Spring applications through testable boundaries, retrieval, tools, evaluation, and operational controls.

SPRING AI | PROMPTS | RAG | TOOLS | EVALUATION

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-29

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `SD 10 — Spring Ecosystem Extensions`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Place Spring AI after the core backend and Spring foundations so AI integration is taught as production software engineering.

Readiness map

Decision	Evidence
START HERE	A Java service calls a model or embedding API; Retrieved context or tool invocation affects correctness; Quality, cost, latency, privacy, and evaluation need explicit controls.
PREPARE FIRST	Spring Boot Book 05; Spring Data Book 06; System-design fundamentals.
MOVE ON WHEN	Understand the planned Spring AI progression; Model AI integration as testable application boundaries; Know the quality and operational questions expected at SDE-2.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

System Design Segment Position

SD 10 - Spring Ecosystem Extensions	**YOU ARE HERE** **SD 11 - Spring AI**	SD 12 - Advanced Java H - Spring

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Spring AI for Java Engineers - Planned Learning Roadmap

> **Publication status:** roadmap edition. Provider-backed examples, retrieval labs, evaluations, and safety exercises will be added after the core Spring service path is complete.

Spring AI provides abstractions and integrations for model clients, prompts, structured output, tools, embeddings, vector stores, retrieval, and observability. This book will focus on engineering contracts around AI features rather than presenting model calls as ordinary deterministic APIs.

Planned sequence

1. Model capabilities, provider differences, tokens, context, latency, cost, and nondeterminism.
2. Spring AI configuration, model clients, prompts, advisors, and structured output.
3. Embeddings, chunking, vector stores, metadata, retrieval, and grounding.
4. Tool calling, permission boundaries, validation, timeouts, and side-effect control.
5. Conversation memory, state ownership, privacy, retention, and multi-tenant isolation.
6. Evaluation datasets, quality metrics, regression tests, and human review.
7. Resilience, rate limits, fallbacks, caching, observability, and cost controls.
8. Prompt injection, data leakage, unsafe output, supply-chain risk, and governance.

Interview focus

The expanded edition will ask readers to design a retrieval-backed Java service, define measurable quality, protect tool execution, handle provider failure, and explain why deterministic unit tests alone cannot validate an AI feature.

Completion gate

A reader is ready to add Spring AI to a system when they can state the data, quality, safety, latency, cost, and fallback contracts; evaluate the feature against representative evidence; and keep privileged actions behind explicit application-controlled authorization.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: model clients and prompts
- Interview Core: structured output, retrieval, and tools
- SDE-2 Follow-up: evaluation, security, cost, and operations

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: SD 10 - Spring Ecosystem Extensions

Series index

Next book: SD 12 - Advanced Java H: Spring Boot and REST APIs

TRY IT | Completion check

[ ] I can recognize the core patterns without being told the data structure or algorithm.
[ ] I can implement the representative Java templates from memory.
[ ] I can explain why each implementation is correct.
[ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
[ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Advanced Java B: Language, OOP, and Modern Java
Java	**JAVA 05**	Advanced Java C: Collections, Streams, and I/O
Java	**JAVA 06**	Advanced Java A: JVM and Execution
Java	**JAVA 07**	Advanced Java D: Concurrency and the Memory Model
Java	**JAVA 08**	Advanced Java E: Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Advanced Java G: Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
System Design	**SD 01**	Advanced Java F: Design, Backend, Testing, and Security
System Design	**SD 02**	MySQL for Java Backend Interviews
System Design	**SD 03**	Hibernate and JPA for Java Backend Interviews
System Design	**SD 04**	Spring Framework for Java Engineers
System Design	**SD 05**	Spring Boot for Java Backend Engineers

SEGMENT	BOOK	FOCUSED LEARNING PATH
System Design	**SD 06**	Spring Data for Java Backend Engineers
System Design	**SD 07**	MongoDB for Java Backend Interviews
System Design	**SD 08**	Redis for Java Backend Interviews
System Design	**SD 09**	Apache Kafka and Spring Kafka
System Design	**SD 10**	Spring Ecosystem Extensions
System Design	**SD 11**	Spring AI for Java Engineers
System Design	**SD 12**	Advanced Java H: Spring Boot and REST APIs
System Design	**SD 13**	Advanced Java I: Persistence, SQL, JPA, and Caching
System Design	**SD 14**	Advanced Java J: Distributed Systems and System Design

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: System Design and Backend - Book 11 of 14 - Study Step 19L. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.